

CSE221: Algorithms

Lab Assignment 01

Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, Java or CPP. But you should choose a specific language for all tasks.
 2. For each task write separate python files like task1.py, task3.py and so on.
 3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
 4. For each problem, take input from files called "inputX.txt", and output at "outputX.txt", where X is the task number. So, for problem 2, in your code, the input file is basically this, "input2.txt". Same for the output file.
 5. Finally zip all the files and rename this zip file as per this format: LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip.
[Example: LabSection01_21101XXX_CSE221LabAssignment01_Fall2023.zip]
 6. You MUST follow all the guidelines, and naming/file/ziping convention stated above. **Failure to do so will result in a straight 50% mark deduction.**
 7. Don't copy from your friends. **For plagiarism you will get 0.**
-

Task 1:

- a) You are given a file "**input1a.txt**". The first line of the input file will contain an integer **T**, representing the number of test cases. The next T lines will contain an integer **N**. You have to calculate if the number is Odd or Even. For each test case print the expected output. All the results must be compiled in a single file, "**output1a.txt**."

Sample Input File	Sample Output File
5 10 19 7 3 100	10 is an Even number. 19 is an Odd number. 7 is an Odd number. 3 is an Odd number. 100 is an Even number.

- b) You are given a file “**input1b.txt**”. The first line of the input file will contain an integer **T**, representing the number of test cases. The next T lines will contain a single arithmetic expression. Each arithmetic expression will start with the prefix “calculate”. It is guaranteed that the expression will have exactly two operands and one operator.

Calculate the result of each expression. Your output format should exactly match with the sample output format. All the results must be compiled in a single file, “**output1b.txt**.”

Sample Input File	Sample Output File
15 calculate 67 + 41 calculate 85 / 5 calculate 13 - 56 calculate 99 - 95 calculate 3 / 10 calculate 12 * 19 calculate 14 - 6 calculate 3 * 88 calculate 45 * 68 calculate 81 - 0 calculate 77 + 40 calculate 8 * 84 calculate 73 - 22 calculate 85 - 86 calculate 28 * 58	The result of 67 + 41 is 108 The result of 85 / 5 is 17.0 The result of 13 - 56 is -43 The result of 99 - 95 is 4 The result of 3 / 10 is 0.3 The result of 12 * 19 is 228 The result of 14 - 6 is 8 The result of 3 * 88 is 264 The result of 45 * 68 is 3060 The result of 81 - 0 is 81 The result of 77 + 40 is 117 The result of 8 * 84 is 672 The result of 73 - 22 is 51 The result of 85 - 86 is -1 The result of 28 * 58 is 1624

Task 2:

Here is the code of bubble sort. Its run time complexity is $\theta(n^2)$. Change the code in a way so that its time complexity is $\theta(n)$ for the **best-case** scenario.

You have to **explain how you have achieved the $\theta(n)$ for the best-case scenario in a comment block of your code.**

```
def bubbleSort(arr):  
    for i in range(len(arr)-1):  
        for j in range(len(arr)-i-1):  
            if arr[j] > arr[j+1]:  
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

Input 1: 5 3 2 1 4 5	Output 1: 1 2 3 4 5
Input 2: 6 5 10 15 20 25 30	Output 2: 5 10 15 20 25 30

For the input 2, your code should run at $\theta(n)$.

Please note, you have to take the input from an input2.txt file, and show the output in an output2.txt file.

Task 3:

Suppose you are given a task to rank the students. You have gotten the marks and id of the students. Now your task is to rank the students based on their marks using a sorting algorithm.

However, you have to keep in mind that your sorting algorithms perform the minimum number of swapping operations.

Input:

The first line of the input file will contain an integer **N** ($1 \leq N \leq 1000$).

The second line will contain N integers, representing the Student ID, S_i ($1 \leq S_i \leq 1000$).

The next line will contain the N integer, S_m ($1 \leq S_m \leq 1000$), which denotes the obtained mark of the corresponding students.

Output:

You have to show the Student Id and obtained marks in descending order based on their obtained mark. If two or more students get the same mark, then students with the lower ID will get prioritized. See the input and output for a better understanding.

Input 1: 7 7 4 9 3 2 5 1 40 50 50 20 10 10 10	Output 1: ID: 4 Mark: 50 ID: 9 Mark: 50 ID: 7 Mark: 40 ID: 3 Mark: 20 ID: 1 Mark: 10 ID: 2 Mark: 10 ID: 5 Mark: 10
Input 2: 4 7 2 5 3 80 60 80 50	Output 2: ID: 5 Mark: 80 ID: 7 Mark: 80 ID: 2 Mark: 60 ID: 3 Mark: 50

Please note, you have to take the input from an input3.txt file, and show the output in an output3.txt file

Task 4:

You have been recently recruited as the Software Engineer at Jumanji Railway Software System. You have a big task at hand. You will be given the N ($1 \leq N \leq 100$) schedule of the train. The next N line will contain the name of the train and the departure time. See the input format for better understanding.

Your task is to write a sorting algorithm that will group the trains in the lexicographical order based on the name of the trains. If two or more trains have the same name, then

the train with the latest departure time will get prioritized. If there is still a tie, then the train which comes first in the input file will come first.

Sample Input	Sample Output
13 ABCD will departure for Mymensingh at 00:30 DhumketuExpress will departure for Chittagong at 02:30 SubornoExpress will departure for Chittagong at 14:30 ABC will departure for Dhaka at 17:30 ShonarBangla will departure for Dhaka at 12:30 SubornoExpress will departure for Rajshahi at 14:30 ABCD will departure for Chittagong at 01:00 SubornoExpress will departure for Dhaka at 11:30 ABC will departure for Barisal at 03:00 PadmaExpress will departure for Chittagong at 20:30 ABC will departure for Khulna at 03:00 ABCE will departure for Sylhet at 23:05 PadmaExpress will departure for Dhaka at 19:30	ABC will departure for Dhaka at 17:30 ABC will departure for Barisal at 03:00 ABC will departure for Khulna at 03:00 ABCD will departure for Chittagong at 01:00 ABCD will departure for Mymensingh at 00:30 ABCE will departure for Sylhet at 23:05 DhumketuExpress will departure for Chittagong at 02:30 PadmaExpress will departure for Chittagong at 20:30 PadmaExpress will departure for Dhaka at 19:30 ShonarBangla will departure for Dhaka at 12:30 SubornoExpress will departure for Chittagong at 14:30 SubornoExpress will departure for Rajshahi at 14:30 SubornoExpress will departure for Dhaka at 11:30

Please note, you have to take the input from an input4.txt file, and show the output in an output4.txt file.

CSE221

Lab Assignment 02

Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, Java or CPP. But you should choose a specific language for all tasks.
2. For each task write separate python files like task1.py, task3.py and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called "inputX.txt", and output at "outputX.txt", where X is the task number. So, for problem 2, in your code, the input file is basically this, "input2.txt". Same for the output file.
5. Finally zip all the files and rename this zip file as per this format:
LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip.
[Example:
LabSection01_21101XXX_CSE221LabAssignment01_Fall2023.zip]
6. You MUST follow all the guidelines, and naming/file/zippping convention stated above. Failure to do so will result in a straight 50% mark deduction.
7. Don't copy from your friends. For plagiarism you will get 0.

Task 01: [Points: 15]

Your little brother, Alice, is very fond of playing with integers. One day, Alice was given a sorted list of **N** integers in ascending order by his school teacher. Now, your brother wants to play a game with you.

Alice will give you an integer, **S**. You have to find if it is possible to find two values from the list (at distinct positions) whose sum is equal to S.

Now you are feeling very tired. So you decided to write a code, so that it can give you the answer very quickly.

- 1) Can you write an $O(N^2)$ Solution to solve the problem? **[Points 5]**
- 2) Come up with an $O(N)$ solution. **[Points 10]**

Input

The first line contains two integers N and S ($1 \leq N \leq 10^5$, $1 \leq S \leq 10^9$), denoting the length of the list, and the target Sum.

In the next line, there will be N integers $a_1, a_2, \dots, a_N$ ($1 \leq a_i \leq 10^9$) separated by space.

Output

Print two integers: **the positions** of the values [**1 based indexing**]. If there are several solutions, you may print any of them. If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
4 10 1 3 5 7	2 4
Sample Input 2	Sample Output 2
6 18 1 5 8 9 9 10	3 6 [4 5 is also a valid answer] [print only one output]

Sample Input 3	Sample Output 3
4 7 2 4 6 8	IMPOSSIBLE
Sample Input 4	Sample Output 4
4 10 1 5 6 8	IMPOSSIBLE

Task 02: [Points: 15]

Alice and Bob are two friends. Alice has a sorted list in ascending order of length **N**. On the other hand, Bob has a sorted list of length **M**. Now, they want to make a sorted list of **N+M** length in ascending order. However, they are not very good at algorithms. Hence, they asked for your help.

Since you are a computer science student, your task is to come up with an efficient algorithm. In the following, let $n = N+M$.

- 1) Find a solution which runs in $O(n \log n)$. **[Points 5]**
- 2) Come up with a solution which runs in $O(n)$. **[Points 10]**

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's sorted list. In the next line, there will be N integers separated by space.

The third line contains another integer M ($1 \leq M \leq 10^5$), denoting the length of Bob's sorted list. In the next line, there will be M integers separated by space.

All the numbers given in the input will fit in a 32-bit signed integer.

It is guaranteed that the given lists will be in sorted order.

Output:

You have to make a sorted list in ascending order from the given lists in ascending order and show the output.

Sample Input/Output:

Sample Input 1	Sample Output 1
4 1 3 5 7 4 2 2 4 8	1 2 2 3 4 5 7 8
Sample Input 2	Sample Output 2
3 2 10 12 6 3 4 6 7 8 9	2 3 4 6 7 8 9 10 12
Sample Input 3	Sample Output 3
4 1 2 3 4 1 10	1 2 3 4 10
Sample Input 4	Sample Output 4
7 2 3 8 8 10 12 14 9 1 1 4 5 6 8 13 15 16	1 1 2 3 4 5 6 8 8 8 10 12 13 14 15 16

Task 3 [10 Marks]

You are a busy person with lots of tasks to do. You have a schedule of tasks represented by intervals of time, where each interval represents a task that you need to complete. However, you can only work on one task at a time, and you want to complete as many tasks as possible.

Given a list of N intervals of time, your task is to determine the maximum number of tasks you can complete and which tasks they are.

Input

The input consists of a single integer N ($1 \leq N \leq 10^5$), the number of tasks, followed by N lines representing the tasks. Each task is represented by two integers S_i and E_i ($0 \leq S_i \leq E_i \leq 10^9$), the start and end times of the task, respectively.

Output

Output a single integer k , the maximum number of tasks you can complete, followed by a line with k intervals of the tasks you can complete.

If there are multiple solutions with the same maximum number of tasks, print any one of them.

Sample Input/Output:

Sample Input 1	Sample Output 1
6 1 3 2 5 3 7 4 6 6 8 7 9	3 1 3 4 6 6 8
Sample Input 2	Sample Output 2
5 1 4 2 5 6 7 4 8 3 6	2 1 4 6 7

Sample Input 3	Sample Output 3
7 0 4 3 4 1 5 9 10 6 9 2 3 1 2	5 1 2 2 3 3 4 6 9 9 10

Task 4 [10 Marks]

Given N tasks and M people, where each task has a start time and end time, implement a greedy algorithm to find the maximum number of tasks that can be completed by M people.

Each task can only be completed by one person and a person can only be assigned one task at a time. Two tasks cannot be completed simultaneously by the same person.

Input

The input consists of two integers N and M ($1 \leq N, M \leq 10^3$), the number of activities and the number of people, respectively. This is followed by N lines representing the activities. Each line contains two integers S_i and E_i ($0 \leq S_i \leq E_i \leq 10^9$), representing the start and end times of the activity, respectively.

Output

Output a single integer representing the maximum number of activities that can be completed.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 2 1 5 3 6 2 5 8 10 6 9	4
Sample Input 2	Sample Output 2
5 2 1 4 2 5 6 7 4 8 3 6	4
Sample Input 3	Sample Output 3
5 2 1 10 2 10 6 7 4 8 3 6	3
Sample Input 4	Sample Output 4
8 3 5 7 2 4 6 8 8 10 1 3 7 9 3 5 2 6	8

Sample Input Explanation:

In sample input 2-

Person 1 will complete the tasks: 1-4, 4-8

Person 2 will complete the tasks: 2-5, 6-7

Brainstroming:

There is no time constraint for this task 4. However, can you solve the problem in $O(n \log n)$?

CSE221
Lab Assignment 03
Fall 2023

Submission Guidelines:

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task1.py, task2.py, and so on.
3. For each problem, take input from files called "**inputX.txt**" and output at "**outputX.txt**", where X is the task number.
4. Add a hand written explanation of 3-4 lines for each of your solutions in a separate document. You may compile all of your explanations in a single file.
5. Finally zip all the files and rename this zip file as per this format: **LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example: **LabSection01_21101XXX_CSE221LabAssignment03_Fall2023.zip**]
6. Don't copy from your friends.
7. You MUST follow all the guidelines, naming/file/zippping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

Task 01: [Points 10]

In this problem, you will be given a list of numbers. You have to sort the list using the Merge Sort algorithm.

Pseudocode of Merge Sort Algorithm:

```
def merge(a, b):
    # write your code here
    # Here a and b are two sorted list
    # merge function will return a sorted list after merging a and b

def mergeSort(arr):
    if len(arr) <= 1:
        return arr
```

```

else:
    mid = len(arr)//2
    a1 = mergeSort(.....) # write the parameter
    a2 = mergeSort(.....) # write the parameter
    return merge(a1, a2)    # complete the merge function above

```

Note: You already have coded the merge function. Do you know in which task?

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's sorted list. In the next line, there will be N integers separated by space.

Output:

You have to sort the number using the Merge Sort algorithm and show the sorted list.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 9 5 4 6 1 3 2 9	1 2 3 4 5 6 9 9
Sample Input 2	Sample Output 2
1 10	10
Sample Input 3	Sample Output 3
6 8 1 4 2 1 3	1 1 2 3 4 8
Sample Input 4	Sample Output 4
7 7 6 5 4 3 2 1	1 2 3 4 5 6 7

Task 02 [10 Points]:

Alice and you are playing with a list of **N** non negative integers. Today you will try to find out the maximum number of the list. Alice writes the following code to find the maximum number.

```
maxValue = arr[0]
for i in range(1,N):
    if maxValue < arr[i]:
        maxValue = arr[i]
```

Recently you have learned merge sort. Now, you are thinking if you can use the divide and conquer approach to find out the maximum from the given list.

Please note, you are not allowed to sort the given list. The motive for this task is not sorting but to use the concepts of divide and conquer approach.

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's list. In the next line, there will be N integers separated by space.

Output:

You have to find out the maximum value from the list using the divide and conquer approach.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 1 7 13 4 5 7 13 12	13
Sample Input 2	Sample Output 2

7 5 15 2 3 10 1 9	15
Sample Input 3	Sample Output 3
1 9	9
Sample Input 4	Sample Output 4
6 5 2 3 10 1 9	10

What is the time complexity of your code?

Task 03 [10 Points]:

Somewhere in the universe, the Biannual Regional Alien Competition is taking place.

There are **N** aliens standing in a line. You will be given a permutation of N, which denotes the height of each alien. A sequence of N numbers is called a permutation if it contains all integers from 1 to N exactly once. For example, the sequences [3,1,4,2], [1] and [2,1] are permutations, but [1,2,1], [0,1] and [1,3,4] – are not.

In the competition, for each alien, the judge wants to count how many aliens are standing on its right side with a strictly smaller height. Then the judge wants to add up all the counts. To do this, the judge writes the following piece of code.

```
count = 0
for i in range(n):
    for j in range(i+1,n):
        if H[i] > H[j]:
            count+=1
```

However, their algorithm wasn't efficient at all. Hence, the alien calls you to write a better solution for the program.

More formally, you have to count how many pairs of aliens are standing in the line such that $H[i] > H[j]$ and $i < j$. Here, A is a permutation of the aliens' heights. And i, j denote the Aliens' positions.

Input

The first line contains a single integer $1 \leq N \leq 10^6$ - the number of total aliens.

The next line contains N integers $H_1, H_2, \dots, H_n (1 \leq H_i \leq N)$ - the height of the i -th alien. It is guaranteed that the given heights will be a permutation of N .

Output

Print a single integer, which denotes the total number of pairs (i, j) such that $i < j$ and $H_i > H_j$.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 1 2 3 4 5	0
Sample Input 2	Sample Output 2
5 5 4 3 2 1	10
Sample Input 3	Sample Output 3
8 2 7 4 1 5 6 8 3	11

Sample Input 3 Explanation:

In the sample input 3, the following pairs on alien's heights satisfy the condition: (2,1), (7,4), (7,1), (7,5), (7,6), (7,3), (4,1), (4,3), (5,3), (6,3), (8,3)

Task 04 [10 points]

You are given a list A of N integers. You have to choose two indices i and j such that $1 \leq i < j \leq N$ and $A[i] + A[j]^2$ is maximum possible. Here, we are considering 1-based indexing.

Write a code which will find the maximum value of $A[i] + A[j]^2$ in $O(N \log N)$.

Input

The first line contains a single integer $1 \leq N \leq 10^6$ - the length of the list.

The next line contains N integers $A_1, A_2, \dots, A_n$ ($-10^8 \leq A_i \leq 10^8$) separated by a space.

Output

Print a single integer - which denotes the maximum possible value of $A[i] + A[j]^2$.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 9 6 5 8 2	73
Sample Input 2	Sample Output 2

8 5 10 4 -3 1 6 -10 2	110
Sample Input 3	Sample Output 3
7 -5 -2 -6 -7 -1 8 2	63

Task 05 [10 Points]

In this problem, you will be given a list of numbers. You have to sort the list using the Quick Sort algorithm in ascending order.

Pseudocode of Quick Sort Algorithm:

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )

```

PARTITION(A, p, r)

```

1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6          exchange  $A[i]$  with  $A[j]$ 
7  exchange  $A[i + 1]$  with  $A[r]$ 
8  return  $i + 1$ 

```

[The code snippet has been taken from the book: Introduction to Algorithms]

Input

The first line contains an integer N ($1 \leq N \leq 10^5$), denoting the length of Alice's list. In the next line, there will be N integers separated by space.

Output:

You have to sort the number using the Quick Sort algorithm in ascending order and show the sorted list.

Sample Input/Output:

Sample Input 1	Sample Output 1
8 9 5 4 6 1 3 2 9	1 2 3 4 5 6 9 9
Sample Input 2	Sample Output 2
1 10	10
Sample Input 3	Sample Output 3
6 8 1 4 2 1 3	1 1 2 3 4 8
Sample Input 4	Sample Output 4
7 7 6 5 4 3 2 1	1 2 3 4 5 6 7

Task 06 [10 Points]

In this problem, you will be given a list of numbers. You have to find the k -th smallest value from the list without sorting using the Partition function of Quick sort.

We will consider the 1-based indexing of the list.

Input

The first line contains an integer N ($1 \leq N \leq 10^6$), denoting the length of the list.

The next line contains N integers $A_1, A_2, \dots, A_n$ ($1 \leq A_i \leq 10^6$) separated by a space.

The third line contains a single integer Q ($1 \leq Q \leq 100$) - which denotes the number of queries you have to answer.

Each of the next Q lines will contain a single integer K ($1 \leq K \leq N$).

Output:

For each query, you have to find the K -th smallest number from the given list.

Sample Input/Output:

Sample Input 1	Sample Output 1
9 // Total Elements 10 11 10 6 7 9 8 15 2 4 // Total queries 5 3 2 7	9 7 6 10

CSE221

Lab Assignment 04

Fall 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called **"inputX_Y.txt"** and output at **"outputX_Y.txt"**, where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this
format: **LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example: **LabSection01_21101XXX_CSE221LabAssignment05_Fall2023.zip**]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

You can find all the Input Files of this lab from here:

<https://drive.google.com/drive/folders/10grRR3iRIfVn6kFLyD86-n8xQewUNU9n?usp=sharing>

Task 01 (Graph Representation):

A) You are given a **directed weighted** Graph, G . The graph consists of N vertices and M edges. In this problem, you have to store the graph using Adjacency Matrix and print the matrix.

Input:

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of vertices and the total number of edges.
The next M lines will contain three integers u_i , v_i , and w_i ($1 \leq u_i, v_i \leq N$, $1 \leq w_i \leq 1000$) – denoting there is an edge between node u_i and v_i with cost w_i .

Output:

You have to make the graph using Adjacency Matrix, and print the Adjacency Matrix in output.

Sample Input/Output:

Sample Input 1	Sample Output 1
4 3 1 3 8 3 2 5 1 4 2	0 0 0 0 0 0 0 0 8 2 0 0 0 0 0 0 0 5 0 0 0 0 0 0 0
Sample Input 2	Sample Output 2
6 7 1 5 6 6 3 5 1 3 9 3 4 7 4 6 1 5 6 8 6 1 6	0 0 0 0 0 0 0 0 0 0 9 0 6 0 0 0 0 0 0 0 0 0 0 0 0 7 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 8 0 6 0 5 0 0 0

B) Previously, we have learned how to represent a graph using the Adjacency matrix. Now in this problem, using the information in Task 01 (a), you have to **build the graph using Adjacency List.**

Sample Input/Output:

Sample Input 1	Sample Output 1
4 3 1 3 8 3 2 5 1 4 2	0 : 1 : (3,8) (4,2) 2 : 3 : (2,5) 4 :
Sample Input 2	Sample Output 2
6 7 1 5 6 6 3 5 1 3 9 3 4 7 4 6 1 5 6 8 6 1 6	0 : 1 : (5,6) (3,9) 2 : 3 : (4,7) 4 : (6,1) 5 : (6,8) 6 : (3,5) (1,6)
Sample Input 3	Sample Output 3
9 15 1 9 4 8 7 5 5 4 7 4 3 2 2 1 10 3 1 15 7 1 13 3 9 8 3 8 4 3 7 3 5 3 1 5 8 2 9 4 8 9 2 10 9 2 12	0 : 1 : (9,4) 2 : (1,10) 3 : (1,15) (9,8) (8,4) (7,3) 4 : (3,2) 5 : (4,7) (3,1) (8,2) 6 : 7 : (1,13) 8 : (7,5) 9 : (4,8) (2,10) (2,12)

Brainstorming: [No need to submit the following questions]

- a) What if it was an **undirected** graph? Can you think of where you have to modify in your code for an undirected graph?
- b) In the last input of Task 01 (B), you notice there are two edges from node '9' to node '2' of cost 10 and 12, known as **parallel edges**. If a graph contains parallel edges, then can we represent the graph using Adjacency Matrix?

Task 02 (Graph Traversal Using BFS):

You have most probably heard of Dora The Explorer. She likes to travel from one country to another.

Currently, Dora is at Doraland. Doraland is a beautiful country, consisting of **N cities** and **M bidirectional roads**.

Recently, Dora has started to learn Graph Algorithms. She knows about BFS and DFS. However, since Dora is still learning, she is not feeling so confident and asks for your help.

Dora contacts you and gives you all the necessary information about Doraland. **Dora will start her journey from city 1**. Now, your task will be **to help Dora to find a path which is similar to the path of BFS graph traversal**.

Input

The given map will be an undirected and unweighted graph.

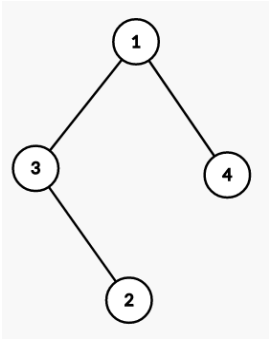
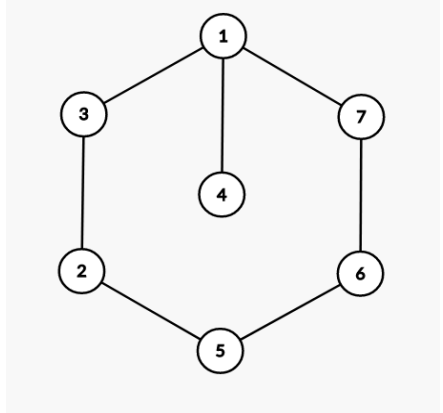
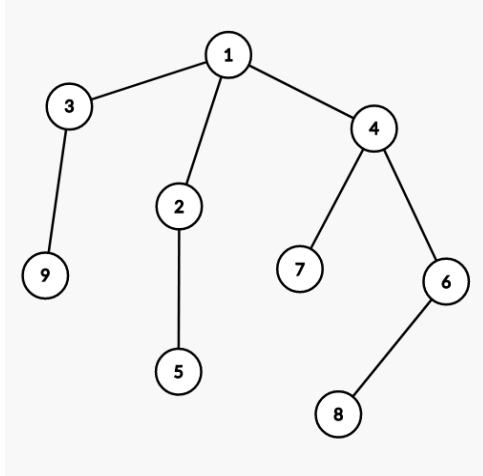
The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the BFS path traversal which Dora will follow to explore the city, starting from city 1.

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

<pre> 4 3 1 3 3 2 1 4 </pre>	<pre> 1 3 4 2 (Another valid path: 1 4 3 2) </pre>	
Sample Input 2	Sample Output 2	Sample Graph 2
<pre> 7 7 1 3 3 2 1 4 2 5 5 6 1 7 7 6 </pre>	<pre> 1 3 4 7 2 6 5 </pre>	
Sample Input 3	Sample Output 3	Sample Graph 3
<pre> 9 8 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9 </pre>	<pre> 1 2 3 4 5 9 6 7 8 </pre>	
Sample Input 4	Sample Output 4	Sample Graph 4

11 14 1 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11	1 2 3 8 10 5 9 7 6 4 11	
--	-------------------------	--

After successful BFS traversal, Dora wants to visit the whole country again. However, this time Dora wants to traverse the country following the path which is similar to the path of DFS graph traversal.

Yes, you guessed it correctly. You have to help Dora this time also.

The input format remains the same as Task 02. **Dora will start her journey from city 1 again.**

Input

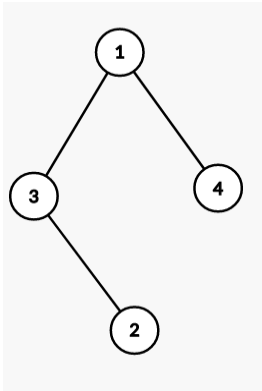
The given map will be an undirected and unweighted graph.

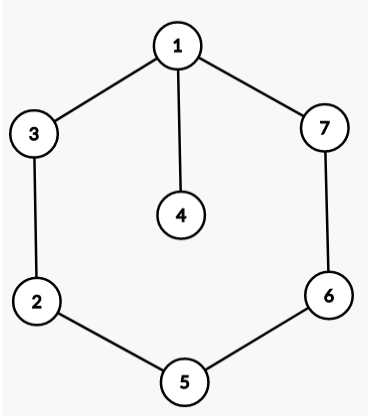
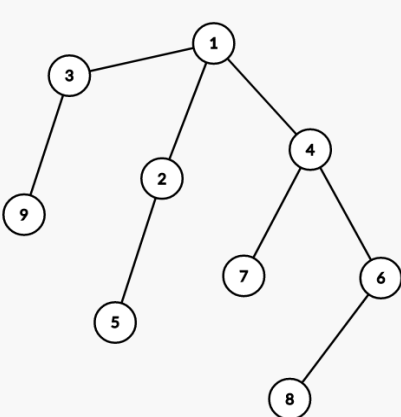
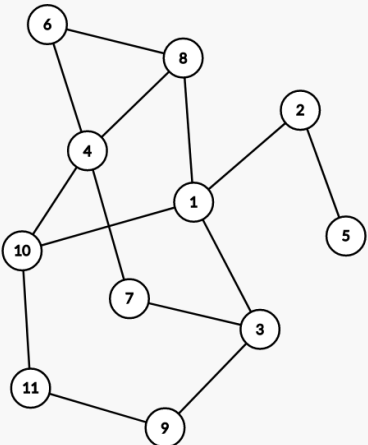
The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the DFS path traversal which Dora will follow to explore the city, starting from city 1.

Sample Input 1	Sample Output 1	Sample Graph 1
<pre> 4 3 1 3 3 2 1 4 </pre>	<pre> 1 3 2 4 (Another valid path: 1 4 3 2) </pre>	
Sample Input 2	Sample Output 2	Sample Graph 2

<pre> 7 7 1 3 3 2 1 4 2 5 5 6 1 7 7 6 </pre>	<pre> 1 3 2 5 6 7 4 </pre>	
Sample Input 3	Sample Output 3	Sample Graph 3
<pre> 9 8 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9 </pre>	<pre> 1 2 5 3 9 4 6 8 7 </pre>	
Sample Input 4	Sample Output 4	Sample Graph 4
<pre> 11 14 1 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11 </pre>	<pre> 1 2 5 3 9 11 10 4 6 8 7 </pre>	

This problem may have multiple valid solutions. However, your solution should follow the dfs path traversal order.

Pseudocode of DFS:

The depth-first-search procedure DFS below assumes that the input graph $G = (V, E)$ is represented using adjacency lists.

```
colourInitializing(G):
1 for each vertex u in G.V:
2     u.colour = 0

DFS(G,u):
1     u.colour = 1
2     for each v in G.Adj[u]:
3         if v.colour == 0:
4             DFS(G,v)
```

Task 04 (Cycle Finding):

It was a very hectic day for Dora. Before going to sleep, Dora wants to find out if there is any Cycle in the map of the city.

Since Dora has traveled the whole country twice, she is feeling very exhausted and asks you to solve the problem. However, Dora has made the roads of the cities **directed** in her map, so that you don't get bored.

In graph theory, a cycle in a graph is a non-empty trail in which only the first and last vertices are equal. [Wikipedia]

Input:

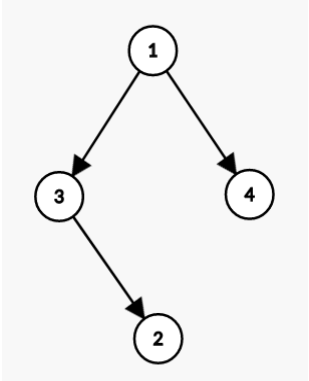
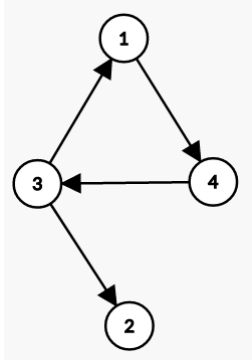
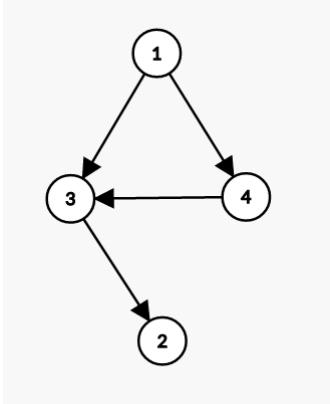
The given map will be a directed and unweighted graph.

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of cities and the total number of roads.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output:

Print "YES" if the map contains any Cycle, otherwise print "NO".

Sample Input 1	Sample Output 1	Sample Graph 1
4 3 1 3 3 2 1 4	NO	
Sample Input 2	Sample Output 2	Sample Graph 2
4 4 3 1 3 2 1 4 4 3	YES	
Sample Input 3	Sample Output 3	Sample Graph 3
4 4 1 3 3 2 1 4 4 3	NO	
Sample Input 4	Sample Input 4	Sample Graph 4

<pre> 9 10 1 2 2 3 3 4 4 5 3 5 1 6 2 7 5 8 5 9 8 9 </pre>	NO	<pre> graph TD 1((1)) --> 2((2)) 1((1)) --> 6((6)) 2((2)) --> 7((7)) 2((2)) --> 3((3)) 3((3)) --> 4((4)) 3((3)) --> 5((5)) 4((4)) --> 5((5)) 5((5)) --> 8((8)) 5((5)) --> 9((9)) 8((8)) --> 9((9)) </pre>
Sample Input 5	Sample Input 5	Sample Graph 5
<pre> 5 7 1 2 2 3 1 3 3 4 4 5 3 5 4 2 </pre>	YES	<pre> graph TD 1((1)) --> 2((2)) 1((1)) --> 3((3)) 2((2)) --> 3((3)) 3((3)) --> 4((4)) 3((3)) --> 5((5)) 4((4)) --> 2((2)) 5((5)) --> 4((4)) </pre>

Brainstorming: [No need to submit the following questions]

- Suppose that you have an undirected and unweighted graph, can you find out if the given graph is a tree or not?
- Suppose that you have an undirected and unweighted graph. Can you find out if the graph contains an Odd length Cycle? An odd length cycle means the cycle will contain odd numbers of vertices.

Task 05 (Find the shortest path):

The next day, Dora again called you for helping her. Among all the cities she likes city 'D' the most.

Since you are becoming very much annoyed with Dora, you have planned to reach city D spending the minimum amount of time.

In the belugaland, moving between two cities connected by a road takes one second. You have to find the minimum time to reach D and show the path in the output.

Dora and you will start your journey from city 1.

Input

The given map will be an undirected and unweighted graph.

The first line contains three integers N , M , and D ($1 \leq N, M, D \leq 100$) – the number of cities, the total number of roads and the destination city.

The next M lines will contain two integers u_i , v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print the minimum amount of time to reach city D from city 1.
Next, print the shortest path you will follow.

Sample Input 1	Sample Output 1
4 3 2 1 3 3 2 1 4	Time: 2 Shortest Path: 1 3 2
Sample Input 2	Sample Output 2
6 6 5 1 3 3 2 1 4 2 6 5 6 4 6	Time: 3 Shortest Path: 1 4 6 5
Sample Input 3	Sample Output 3

9 8 7 1 2 1 3 1 4 2 5 4 6 4 7 6 8 3 9	Time: 2 Shortest Path: 1 4 7
Sample Input 4	Sample Output 4
11 14 5 4 2 1 3 10 4 2 5 4 6 4 7 6 8 3 9 3 7 1 8 4 8 1 10 9 11 10 11	Time: 4 Shortest Path: 1 8 4 2 5
Sample Input 5	Sample Output 5
4 3 1 1 3 3 2 1 4	Time: 0 Shortest Path: 1

This problem may have multiple valid shortest paths. However, the minimum time required to reach the destination should be matched exactly with the output.

Task 06 (Flood Fill):

Dora is leaving the country. Before leaving the country, as a token of gesture, Dora gifted you with a map of the jungle "Jumanji".

The map is very interesting. The map is a 2D grid. Some of those cells are occupied by diamonds and obstacles. [See the Sample input for better understanding.]

Now, you are on a journey to Jumanji in search of Diamonds. However, since the jungle is full of ferocious creatures, you can't collect all the diamonds. **You will choose only one start position such that you collect the maximum amount of diamonds. Please note that you can not move to any cell which contains obstacles.**

Input

The first line contains two integers R and H ($1 \leq R, H \leq 100$) – the number of rows and columns respectively in the grid.

The next R line will contain H characters. Each character represents the status of a cell as follows.

- 1) '.': Empty Cell → You can move here.
- 2) 'D': Cell with a Diamond → If you move to the cell containing 'D', you will collect that Diamond.
- 3) '#': Cell with an obstacle → You can't move to this cell.

Output

Print a single integer that denotes the maximum amount of Diamonds you can collect.

[The diamonds are coloured red to demonstrate which diamonds have been collected.]

Sample Input 1	Sample Output 1
4 3 .. D D .. .D. ##.	3
Sample Input 2	Sample Output 2
10 7 ...#...D ...#D.. D ...#.D.	11

<pre> D..#... DDD########.. .#D.#DD .####.. DDD...D </pre>	
Sample Input 3	Sample Output 3
<pre> 9 11 .#...D...D.. .#.#####. D#.##..D..#. D#D#.###.#D .#.##..D#.##. .#.#####.#D D#..D...D#. .#####. ...D..D...D </pre>	15
Sample Input 4	Sample Output 4
<pre> 5 5 ####. #D.#. ####. </pre>	1
Sample Input 5	Sample Output 5
<pre> 5 5 ####. #..#. ####. </pre>	0
Sample Input 6	Sample Output 6
<pre> 1 5 D.... </pre>	1
Sample Input 7	Sample Output 7
<pre> 12 12####..... .#D.#..... .####.....###. </pre>	4

<pre> ...D....#D#.#D#. .#####D#. .#D....##.#. .#.D..D##D#. .#####. </pre>	
---	--

Bonus Tasks: Task 07 and Task 08 [5+5 = 10 Points]

Note that these are bonus problems. Even if you do not attempt these problems, you can get a perfect score. So, do not worry if you find it too hard!

The faculty may ask you viva for these problems. If you are unable to give a correct explanation, you will receive 0 in this lab.

Bonus 1: Task 07:

Dora returns to Doraland once again. She finds that the city has undergone significant improvements since her last visit. The city structure has been reconstructed to make it more beautiful. Currently, **Doraland consists of N cities and all the cities are connected with N-1 bidirectional roads.**

However, this time, Dora will have a very short trip and as usual, she seeks your help. According to Dora's wish, you will choose two cities, let's say city A and city B in such a way that while traveling from city A to city B Dora can pass through a maximum number of cities. You have to keep in mind that it is not possible to visit any roads twice in the journey since Dora is on a short trip.

Can you help Dora find such two cities?

Input

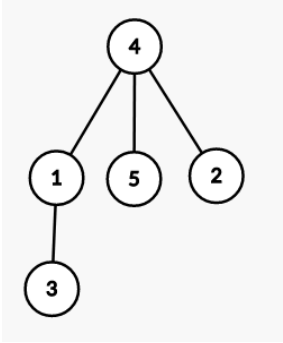
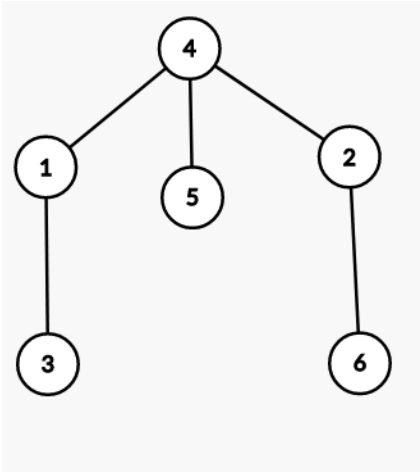
The given map will be an undirected and unweighted graph.

The first line contains one integer N ($2 \leq N \leq 100$) – the number of cities.

The next $N-1$ lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a bidirectional road between u_i and v_i .

Output

Print two cities A and B which satisfy the conditions stated in the problem statement.

Sample Input 1	Sample Output 1	Sample Graph 1
<pre> 5 4 1 4 2 1 3 4 5 </pre>	<pre> 2 3 Or (3 5) [You may print only one valid output] </pre>	 <pre> graph TD 4((4)) --- 1((1)) 4 --- 5((5)) 4 --- 2((2)) 1 --- 3((3)) </pre>
Sample Input 2	Sample Output 2	Sample Graph 2
<pre> 6 4 1 4 2 1 3 4 5 2 6 </pre>	<pre> 3 6 </pre>	 <pre> graph TD 4((4)) --- 1((1)) 4 --- 5((5)) 4 --- 2((2)) 1 --- 3((3)) 2 --- 6((6)) </pre>
Sample Input 3	Sample Output 3	Sample Graph 3

13 1 2 1 3 1 4 4 5 5 6 5 7 7 8 8 9 9 13 6 10 10 11 11 12	12 13	
--	-------	--

This problem may have multiple valid answers. However, make sure it satisfies the given condition.

Explanation:

In test case 1, if you choose city 2 and city 3, then the path will be

$2 \rightarrow 4 \rightarrow 1 \rightarrow 3$. If you choose city 3 and city 5, then the path will be

$3 \rightarrow 1 \rightarrow 4 \rightarrow 5$.

Since, in both paths you can visit a maximum of four cities by using the roads only one, both the answers are valid.

In test case 2, one can see that visiting from city 3 to city 6 (or vice-versa) is only the valid answer. The path will be $3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 6$.

Bonus 2: Task 08:

As mentioned earlier, Dora is on a short trip and she is leaving the country. Before leaving the country, as a token of gesture, Dora gifted you with a problem this time, which you will find in this link in the lightoj, an online judge.

To open the gift, you have to create an account at lightoj first. [<https://lightoj.com/>]

Problem Link: <https://lightoj.com/problem/back-to-underworld>

CSE221

Lab Assignment 05

Fall 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called **"inputX_Y.txt"** and output at **"outputX_Y.txt"**, where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this
format: **LabSectionNo_ID_CSE221LabAssignmentNo_Fall2023.zip**
[Example: **LabSection01_21101XXX_CSE221LabAssignment05_Fall2023.zip**]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

Task 01 [30 points]

We all know about the prerequisite course. For example, before taking CSE221, we have to complete CSE220. And prior to taking CSE220 we need to complete CSE111 and CSE110. The course sequence is as follows:

CSE110 → CSE111 → CSE220 → CSE221

In this problem, there are **N** courses in the curriculum. There are **M** prerequisite requirements of the form "course **A** has to be completed before course **B**".

Your task is to find an order in which you can complete the courses.

A) Solve it using the DFS approach. [15 points]

B) Solve it using the BFS approach. [15 points]

Input:

The first input line has two integers **N** ($1 \leq N \leq 1000$) and **M** ($1 \leq M \leq N^2$) - the number of courses and prerequisite requirements. The courses are numbered 1,2,3,...,N.

Next, there are M lines describing the requirements. Each line has two integers **A, B** ($1 \leq A, B \leq N$)- course A has to be completed before course B.

Output:

Print an order in which you can complete the courses. Please note, there could be multiple correct sequences. You can print any valid order that includes all the courses.

If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
5 3 3 1 1 2 4 5	3 4 1 5 2

Sample Input 2	Sample Output 2
6 6 1 2 2 3 4 3 4 5 5 6 6 4	IMPOSSIBLE
Sample Input 3	Sample Output 3
8 10 1 2 1 4 2 4 2 5 2 3 4 6 4 5 6 5 5 3 7 8	1 7 2 8 4 6 5 3

Task 02[10 points]

The problem statement of this problem is the same as **Task 01**. The only difference is that in this task, you have to find the lexicographically smallest valid course sequence.

If you have two sequences, for example: A: $3 \rightarrow 1 \rightarrow 2 \rightarrow 4$ and B: $3 \rightarrow 1 \rightarrow 4 \rightarrow 2$. Then path A is lexicographically smaller than path B.

Input:

The first input line has two integers **N** ($1 \leq N \leq 1000$) and **M** ($1 \leq M \leq N^2$) - the number of courses and prerequisite requirements. The courses are numbered $1, 2, 3, \dots, N$.

Next, there are M lines describing the requirements. Each line has two integers **A, B** ($1 \leq A, B \leq N$)- course A has to be completed before course B.

Output:

Print the lexicographically smallest valid course sequence in which you can complete the courses.

If there are no solutions, print "IMPOSSIBLE".

Sample Input 1	Sample Output 1
5 3 3 1 1 2 4 5	3 1 2 4 5
Sample Input 2	Sample Output 2
6 6 1 2 2 3 4 3 4 5 5 6 6 4	IMPOSSIBLE
Sample Input 3	Sample Output 3
8 10 1 2 1 4 2 4 2 5 2 3 4 6 4 5 6 5 5 3 7 8	1 2 4 6 5 3 7 8

Task 03[10 points]

You are given a Directed Graph consisting of **N** vertices and **M** edges. You have to find the strongly connected components of the given graph.

Input:

The given map will be a directed and unweighted graph.

The first line contains two integers N and M ($1 \leq N, M \leq 100$) – the number of vertices and the total number of edges.

The next M lines will contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$) – denoting there is a road between u_i to v_i .

Output

Print all the strongly connected components of the given graph. See the output for better understanding.

Sample Input 1	Sample Output 1
5 5 1 2 2 3 2 4 3 1 4 5	1 2 3 4 5
Sample Input 2	Sample Output 2
8 9 1 2 1 6 2 3 3 4 4 5 4 6 5 7 6 2 7 5	1 2 3 4 6 5 7 8
Sample Input 3	Sample Output 3
4 3 1 2	1 2

2 3 2 4	3 4
------------	--------

CSE221

Lab Assignment 06

FALL 2023

Submission Guidelines :

1. You can code all of them either in Python, CPP, or Java. But you should choose a specific language for all tasks.
2. For each task write separate python files like task2.py, task3.py, and so on. For problems that have subproblems, name those like task1A.py, task1B.py, and so on.
3. Add a hand written explanation of 3-4 lines for each of your solutions and submit that as a single document.
4. For each problem, take input from files called "**inputX_Y.txt**" and output at "**outputX_Y.txt**", where X is the task number and Y is the sample i/o number. For example, for problem 2 sample 1, the input file is this, "input2_1.txt". For problems that have subproblems, name the files like "input1a_1.txt", "output1a_1.txt" and so on. Same for output.
5. For each task include at least one input file (if any) in the submission folder.
6. Finally zip all the files and rename this zip file as per this format:**LabSectionNo_ID_CSE221LabAssignmentNo_FALL2023.zip**
[Example:**LabSection01_21101XXX_CSE221LabAssignment06_FALL2023.zip**]
]
7. Don't copy from your friends.
8. You MUST follow all the guidelines, naming/file/zipping convention stated above.

Failure to follow instructions will result in a straight 50% mark deduction.

A useful tool for making graphs:

https://csacademy.com/app/graph_editor/

Task 01:[10 Marks]

You are given a weighted, directed graph with N nodes and M edges. Each edge is represented as a triple (u, v, w) , where u and v are the nodes connected by the edge and w is the weight of the edge.

Your task is to find the shortest path from a source node S to all other nodes in the graph using **Dijkstra's algorithm**. You should output the shortest distance from the source node to each of the other nodes in the graph. If a node is unreachable from the source node, its distance should be represented as -1 .

Input

The first line of the input contains two integers, N and M ($1 \leq N \leq 1000$, $1 \leq M \leq 100000$) denoting the number of nodes and edges in the graph, respectively.

The next M lines each contain three integers, u, v ($1 \leq u, v \leq N$), and w ($1 \leq w \leq 100$) denoting an edge from node u to node v with weight w .

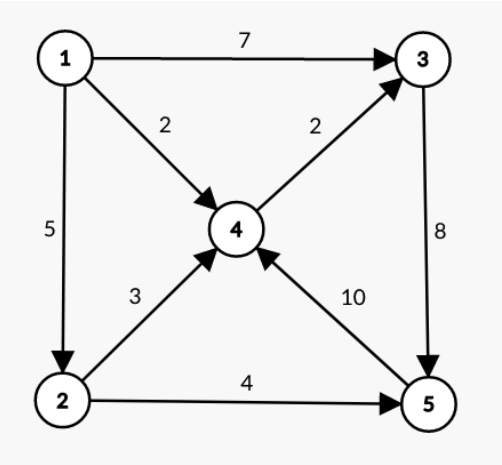
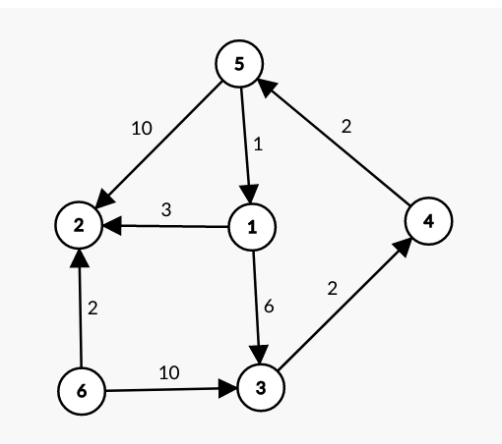
The last line of the input contains an integer S ($1 \leq S \leq N$) denoting the source node.

Output

Output N space-separated integers, where the i -th integer represents the shortest distance from the source node to node i .

If a node is not reachable from the source node, output -1 instead.

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

5 8 1 2 5 1 3 7 1 4 2 2 4 3 2 5 4 3 5 8 4 3 2 5 4 10 1	0 5 4 2 9	
Sample Input 2	Sample Output 2	Sample Graph 2
6 8 1 2 3 1 3 6 3 4 2 4 5 2 5 2 10 5 1 1 6 2 2 6 3 10 3	5 8 0 2 4 -1	

Explanation of Sample Input 1

Using Dijkstra's algorithm, the shortest path from node 1 to all other nodes in the given graph is:

- Distance from node 1 to node 1 is 0.
- Distance from node 1 to node 2 is 5.
- Distance from node 1 to node 3 is 4.
- Distance from node 1 to node 4 is 2.
- Distance from node 1 to node 5 is 9.

Task 02:[10 Marks]

Alice and Bob are in a hurry to meet each other and have to traverse through a directed graph with weighted edges. Alice starts from node S and Bob starts from node T. They want to find a common node in the graph, where they can meet each other in the minimum amount of time. Alice or Bob can wait at any node if they want to.

Input

The first line contains two integers N ($1 \leq N \leq 10000$) and M ($1 \leq M \leq 100000$) separated by a space, denoting the number of nodes and edges in the graph, respectively.

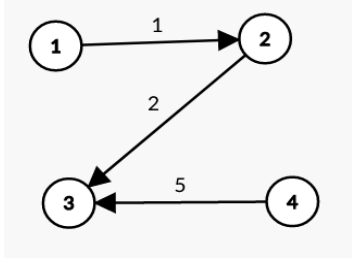
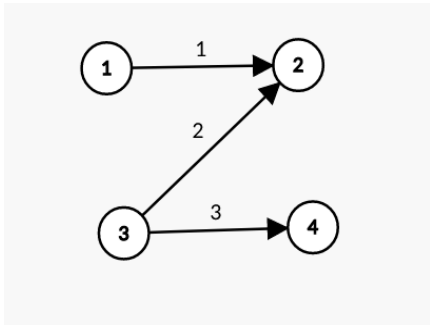
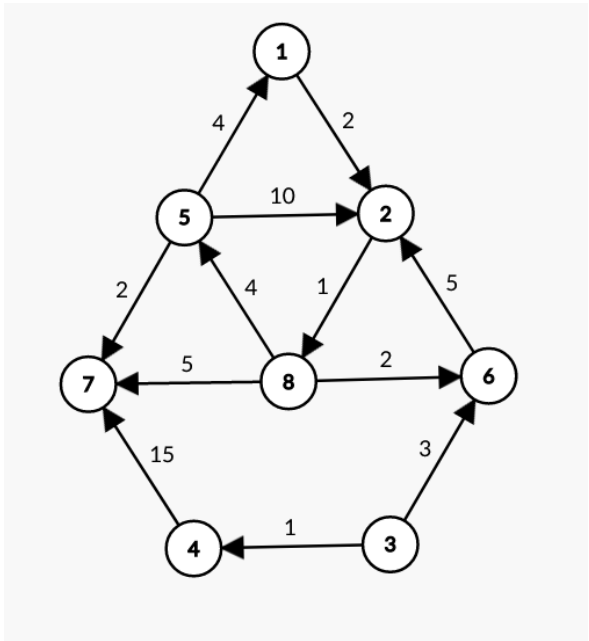
The next M lines each contain three integers, u, v ($1 \leq u, v \leq N$), and w ($1 \leq w \leq 100$) denoting an edge from node u to node v with weight w .

The last line contains two integers S, T ($1 \leq S, T \leq N$) which denotes the starting node of Alice and the starting node of Bob respectively.

Output

Output two integers separated by a space. The first integer is the minimum time it takes for Alice and Bob to meet. The second integer is the vertex where they will meet. If there is no such node, print "Impossible".

Sample Input 1	Sample Output 1	Sample Graph 1
----------------	-----------------	----------------

4 3 1 2 1 2 3 2 4 3 5 1 4	Time 5 Node 3	
Sample Input 2	Sample Output 2	Sample Graph 2
4 3 1 2 1 3 2 2 3 4 3 1 4	Impossible	
Sample Input 3	Sample Output 3	Sample Graph 3
8 12 3 6 3 3 4 1 6 2 5 4 7 15 2 8 1 8 6 2 8 7 5 8 5 4 5 2 10 5 1 4 1 2 2 5 7 2 3 5	Time 8 Node 2	

Task 3 [10 Marks]

Study material: <http://www.shafaetsplanet.com/?p=763>

There is a group of N people living in a small village. They live in their own house. Although they are all neighbors, they don't all know each other very well.

Each person in that village has their own unique identity - labeled with an integer value between 1 to N . Initially, the villagers don't have any friends. As time passes by, they begin to make friendship between themselves.

In this problem, you will be given a description of K friendships. You have to print an integer value which denotes the size of their friend circle.

Suppose, there are five people living in the village labeled with 1,2,3,4 and 5. Initially, the size of each friend circle is one, since no friendship has been created yet.

One day, person 1 and person 2 become friends. So the size of their friend circle becomes two. Next day, person 3 and person 4 become friends and the size of their friendship becomes two as well. After a few days, person 1 and person 4 become friends. Now the size of their friend circle becomes four consisting of persons 1,2,3 and 4.

Input Format:

The input consists of two integers, separated by a space, denoting the number of people in the village, N ($1 \leq N \leq 10^5$) and the number of queries that will follow, K ($1 \leq K \leq 10^5$).

The next K lines contain two integers A_i and B_i each ($1 \leq A_i, B_i \leq N$ and $A_i \neq B_i$), separated by a space, representing two people who have become friends as a result of the query.

Output Format:

For each query, output a single integer on a new line representing the size of the friend circle that the two people belong to after becoming friends.

Sample Input/Output:

Sample Input 1	Sample Output 1
5 3 1 2 3 4 1 4	2 2 4
Sample Input 2	Sample Output 2
8 7 2 4 4 5 3 6 4 7 3 1 2 7 6 2	2 3 2 4 3 4 7

Sample Input Explanation:

In sample input 2,

Query 0: Initially, there are 8 people in the village who do not know each other.

{1} {2} {3} {4} {5} {6} {7} {8}

Query 1: After person 2 and person 4 becoming friends:

{1} {2,4} {3} {5} {6} {7} {8}

The output is 2, since the size of the friends circle {2,4} is 2.

Query 2: After person 4 and person 5 becoming friends:

{1} {2,4,5} {3} {6} {7} {8}

The output is 3, since the size of the friends circle {2,4,5} is 3.

Query 3: After person 3 and person 6 becoming friends:

{1} {2,4,5} {3,6} {7} {8}

The output is 2, since the size of the friends circle {3,6} is 2.

Query 4: After person 4 and person 7 becoming friends:

{1} {2,4,5,7} {3,6} {8}

The output is 4, since the size of the friends circle {2,4,5,7} is 4.

Query 5: After person 3 and person 1 becoming friends:

{2,4,5,7} {1,3,6} {8}

The output is 3, since the size of the friends circle {1,3,6} is 3.

Query 6: Since the person 2 and person 7 are already in the same friend circle, nothing changes:

{2,4,5,7} {1,3,6} {8}

The output is 4, since the size of the friends circle {2,4,5,7} is 4.

Query 7: After person 6 and person 2 becoming friends:

{2,4,5,7,1,3,6} {8}

The output is 7, since the size of the friends circle {2,4,5,7,1,3,6} is 7.

Task 4 [10 Marks]

In the kingdom of Beluga, there are N cities connected by M roads, each with a maintenance cost associated with it. There is at least one path between any two cities. The king is concerned about the increasing maintenance cost and decides to take action.

He calls upon his council, and they suggest that they find a minimum-cost set of roads that connects all cities while minimizing the maintenance cost. Then the king decides to reduce the total maintenance cost by destroying some of the existing roads, instead of building new ones.

Since you are a very good programmer the king calls you. He asks you to find out what the lowest maintenance cost can be achieved after destroying a few roads while ensuring there still exists a path from each city to another.

Input

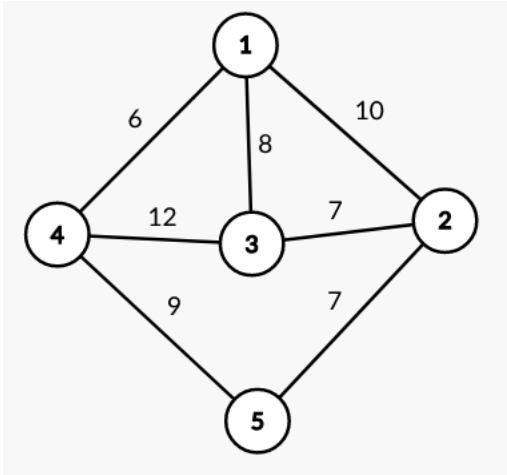
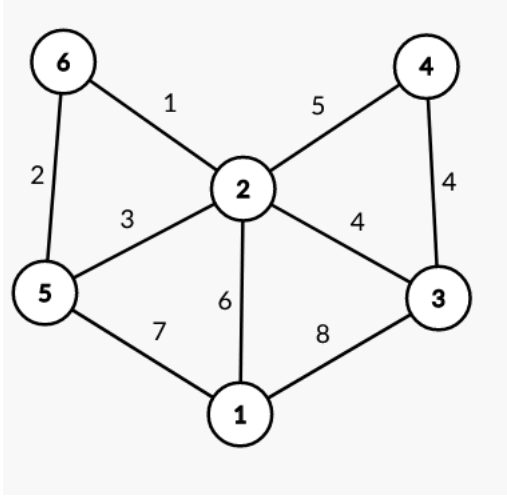
The first line of the input contains two space-separated integers, N and M ($1 \leq N \leq 10^5$, $1 \leq M \leq 10^6$), representing the number of cities and roads in the kingdom of Beluga, respectively.

The next M lines each contain three space-separated integers, u , v , and w ($1 \leq u, v \leq N$, $1 \leq w \leq 10^9$), where u and v denote the endpoints of a road and w represents its maintenance cost.

Output

The output should contain a single integer, the minimum total maintenance cost achievable.

Sample Input/Output:

Sample Input 1	Sample Output 1	Sample Graph 1
5 7 1 2 10 1 3 8 1 4 6 2 3 7 2 5 7 3 4 12 4 5 9	28	 <pre> graph TD 1((1)) --- 10 2((2)) 1 --- 8 3((3)) 1 --- 6 4((4)) 2 --- 7 3 3 --- 12 4 3 --- 7 5((5)) 4 --- 9 5 </pre>
Sample Input 2	Sample Output 2	Sample Graph 2
6 9 1 2 6 2 4 5 2 3 4 1 3 8 4 3 4 2 6 1 2 5 3 5 6 2 5 1 7	17	 <pre> graph TD 1((1)) --- 6 2((2)) 1 --- 8 3((3)) 1 --- 7 5((5)) 2 --- 4 3 2 --- 5 4((4)) 2 --- 1 6((6)) 3 --- 4 4 4 --- 4 5 5 --- 2 6 </pre>

Time complexity, Searching, Sorting

1. Solve the following questions about time complexity.

a. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n / 2; i > 1; i /= 6) {
    for (j = 2; j <= n; j *= 4) {
        for (k = 0; k <= j; k *= 3) {
            p = p + n / 2;
        }
    }
}
```

b. Calculate the time complexity of the following code snippet:

```
int p = 0;
for (i = n; i > 1; i -= 1) {
    for (j = 2; j <= n; j += 1) {
        p = p + n;
        if (p > (n ^ i)) // ^ sign means XOR in code
        {
            break;
        }
    }
}
for (i = n / 2; i > 1; i /= 6) {
    for (j = 2; j <= i; j *= 4) {
        for (k = 0; k <= j; k *= 3) {
            p = p + n / 2;
        }
    }
}
```

2. After working with the binary search algorithm, as a CSE221 student you want to explore its strength. You have several ideas in your mind. Try to modify the algorithm to implement your ideas.

- What if you want to return the first index of the search key in case there are duplicates. For example: your search key is 13 and there are three 13s in the list.
- Now, you also want to return the number of times the key appears in the list. Say, the search key is 54 and it occurs 4 times in indices 6,7,8,9. Then you should return (6,4).

3.

Consider the following list:

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

Will the algorithm given on the right be able to find the search value $T=2$ for the list above?

If yes, **show** the value of L , R , m in each step of the algorithm. If no, **explain** why not.

```
function binary_search(A, n, T) is
    L := 0
    R := n - 1
    while L ≤ R do
        m := floor((L + R) / 2)
        if A[m] < T then
            L := m + 1
        else if A[m] > T then
            R := m - 1
        else:
            return m
    return unsuccessful
```

Here, A denotes the list, and n denotes its size.

4. You are given an array containing N distinct integers in a wave-like sequence. Meaning, the numbers in the beginning are in ascending order, and after a specific position, they are in descending order. For example: [1, 3, 4, 5, 9, 6, 2, -1]
- You have to find the maximum number of this sequence. Can you devise an efficient algorithm such that the time complexity will be less than $O(N)$?
- Present your solution idea as a pseudocode/ python code/ flowchart/ step-by-step instructions/ logical explanation in one-two paragraphs.
 - Write the time complexity of your algorithm.
5. [Mid - Summer '24] Your friend gave you an integer (key) and asked you to find its integer square root without using built-in functions like `sqrt()` or exponentiation. For example, if the input is 25, the output should be 5 because $5^2=25$. If the square root is not an integer, return the floor value of the actual square root. For example, for an input of 10, the output should be 3.
- Your friend's initial approach is to use linear search (time complexity $O(n)$) as shown below.

```
def LinearSearchToFindSquareRoot (key) :
    result = -1
    for i in range(1, key, 1):
        if i * i <= key:
            result = i
    return result
```

- Present an $O(\sqrt{n})$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.
 - Present an $O(\log_2 N)$ time algorithm with pseudocode/programmable code/step-by-step logical instructions.
6. For different tasks, we often need to search for things. And most of the time, we sort the dataset before performing search operations. Once, you found the following list of numbers.
- [2, 5, 1.2, 6.7, 1.7, 9.3, 2.2, 7.7, 0, -4, -5.1, 2, 5, 5.2]
- To search an element in an unsorted array, we need $O(N)$ time using linear search. Binary search works in $O(\log N)$ time but the array needs to be sorted beforehand which takes at least $O(N \log N)$ time in general. Why would you then sort the array first and then perform binary search instead of just performing linear search?
 - Can you modify count sort so that it may work with negative integers as well?
 - Can you modify count sort so that it may work with the given list?
 - Say, you are working in a system where you need to sort a very big dataset. The memory available to you can barely accommodate the data. You have two options – merge sort & quick sort. Which one should you choose? Why?
 - Construct an array where quick sort fails to work in $O(N \log N)$ time.
7. [Mid - Fall '22] Jack loves to play with integers. He created a list of n integers where the even indices hold numbers in decreasing order and the odd indices hold numbers in increasing order. For example, this is a list of $n=8$ integers Jack made.

Index	0	1	2	3	4	5	6	7
Number	23	2	19	3	7	11	5	13

[Explanation:

The indices 1, 3, 5, and 7 have numbers 2, 3, 11, and 13 in increasing order.

The indices 0, 2, 4, and 6 have numbers 23, 19, 7 and 5 in decreasing order.]

- a. Present a linear time algorithm to sort the list in ascending order. Write your algorithm with pseudocode/programmable code/step-by-step logical instructions.

Graph Theory - I

Lecture material:

<https://drive.google.com/file/d/1OkLO91GV4M23e-1EtvFr9yeGWgk6V91q/view?usp=sharing>

For the questions that ask you to propose/present an algorithm, you can write either a programmable code or a pseudocode or step-by-step instructions.

Problem #1:

Answer the following general questions regarding the basic terminology of Graph Theory:

1. In a directed graph, if there is an edge from Node A to Node B, and not the other way around, can A and B be called adjacent/neighbor to each other?
2. In an undirected graph of n vertices, how many edges can we construct without allowing loops (edge to self) and multiple edges between a pair of vertices?
3. Same question for directed graphs.
4. How many edges can we construct in an undirected graph with n vertices without introducing any cycle? Loops can also be considered as cycles.
5. Same question for directed graphs.
6. Take any graph, try to construct a path that starts from a vertex, then visits all the edges of the graph exactly once. It will not be possible in all graphs, only in some having a special property. What's that property?
7. Take any graph, try to construct a path that starts from a vertex, then visits all other vertices of the graph exactly once (some edges might be unvisited). Like the previous question, it will not be possible in all graphs. Just find some examples with both cases.
8. Analyze the efficiency of using adjacency list over adjacency matrix for different types of graphs: undirected, directed, weighted, unweighted, sparse, dense...

Problem #2:

Suppose we have the following social network, Friendsbook where:

Person A is friends with Person B, C, D, and E.

Person B is friends with Person A, C, and F.

Person C is friends with Person A, B, D, and G.

Person D is friends with Person A, C, E, G, and H.

Person E is friends with Person A, D, and H.

Person F is friends with Person B and G.

Person G is friends with Person C, D, F, and H.

Person H is friends with Person D, E, and G.

- A. Draw a graph to represent the network using each user as a node or vertex and the friendships as edges between the nodes.
- B. Create the adjacency list and adjacency matrix for the graph representation

- C. For each pair of users, calculate the number of friends they have in mutual.
- D. In Friendsbook, a user can share posts. Suppose the CEO of Friendsbook hired you for developing their post display system. You were given the task to implement a system that displays posts from users who are at most two degrees of connection away from the one who is scrolling his feed. After looking at the requirements you came up with an idea to implement the system using the BFS algorithm. Now, for person F, simulate your algorithm. Can F see the posts of all other users in his feed?

Problem #3:

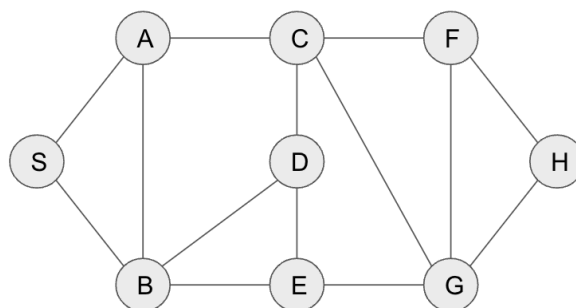
The following information is stored after implementing the DFS algorithm on an undirected graph.

Nodes	1	2	3	4	5	6	7
Parent							
Starting Time	8	11	5	6	4	1	2
Finish Time	9	12	10	7	13	14	3
Distance from Root							

- A. Draw the DFS tree.
- B. Fill up the empty rows of the table.

Problem #4:

Consider this graph:



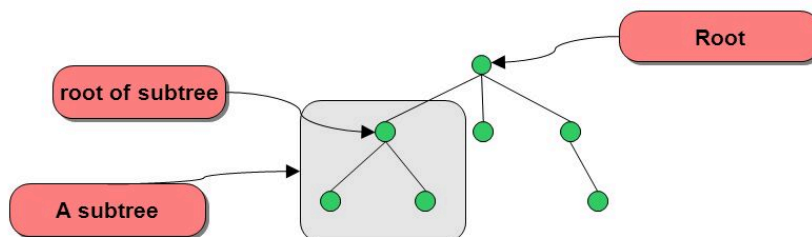
- A. Justify $\sum_{v \in V} \deg(v) = 2m$, where m = number of edges in the graph.
- B. Calculate the number of additional edges that can be added between the nodes randomly, keeping the graph 'simple' (without adding multiple edges between any two nodes, loops in a single node).

Problem #5:

Trees and subtrees



- In a rooted tree, any node is a root of a **subtree** consisting of itself and the nodes "below" it
- By definition there is only one path between the root and each of the other nodes.
 - Because a root is also a node the main property applies
- The convention used in computer science is that roots are drawn on the top. It may seem strange at the beginning but you'll get used to it



Source: <https://slideplayer.com/slide/7322529/>

You are given a Binary Tree with N nodes. That means each of the nodes has zero to two children, and its own subtree. Nodes are numbered from 1 to N .

You will be given Q queries. Each of them tells you a node number P , and asks you to find the size of its subtree (number of nodes in the subtree including P as the root).

- How would you build the adjacency list/matrix of this tree? Show with an example tree.
- You have to answer each query in constant time, $O(1)$. However, you are allowed to do a preprocessing of $O(V+E)$. Propose a suitable algorithm that satisfies this.

Problem #6:

Suppose you are given a directed graph where the number of vertices is V and the number of edges is E . The vertices are numbered from 1 to V . You are given the task to find out whether vertex '1' is reachable from all the other vertices, i.e. there exist paths to go from '2' to '1', '3' to '1', ... and so on.

Propose an $O(V+E)$ algorithm to find the answer to this question.

Clarification: You cannot run DFS from each source. That would take the complexity to $O(V \cdot E)$.

The End

Graph Theory II: Shortest Paths, MST

Deadline: <check announcement in your section's slack channel>

Instructions:

1. Prepare a single pdf file containing your works
 2. Submit through the google form: <check announcement in your section's slack channel>
 3. You need to sign in to your bracu g-suite email account to see the form
-

Assume that you are given a function called $dijkstra(G, a)$ which takes a graph G , and a source vertex a as input, then finds the shortest path (distances, $dist$ and parents, par) from a to every other vertex in the graph. Use this function to solve problems #1 and #2. Present your solution idea with a pseudocode, accompanying necessary explanation.

Problem #1: Optimum meeting place

You, and your friend Benji, live in a city which has 50 areas numbered from 1 to 50. Some of the areas are connected with bi-directional roads of various lengths. Your house is in 1 and Benji's is in 50. Both of you want to meet. Now propose an algorithm to choose the meeting point that minimizes the total travel time. In other words, if the meeting area you chose is X , it has the minimum $dist(1, X) + dist(50, X)$ among all possible area choices.

Expected time complexity: $O(E * \log_2(V))$

Problem #2: Meet in the middle!

Continuing on the scenario of Problem #1, but assuming all the roads are one-way (representing a directed graph)...

Benji now has asked you to visit his house in area 50. He likes KFC, so you are thinking about getting a bucket of fried chicken for him. There are KFC outlets available in K different areas in the city. You have to make a stop at one of them. Now propose an algorithm to choose the area from the available K options that minimizes your total travel time. In other words, if the area you chose is X , it has the minimum $dist(1, X) + dist(X, 50)$ among all possible choices.

Expected time complexity: $O(E * \log_2(V))$

*Note: If you have to run Dijkstra's algorithm from K different sources, the complexity is $O(K * E * \log_2(V))$, which exceeds the expected complexity for this problem.*

Problem #3: Dijkstra's algorithm in a 'negative' weight graph!!

Draw a directed graph as described below.

No. of vertices: 7, No. of edges: 9

List of vertices, V : {1, 2, 3, 4, 5, 6, 7}

List of edges, E [in (u, v, w) format which means there is an outgoing edge from u to v with weight w] :

{(1, 2, 8), (2, 3, - 8), (1, 3, 1) (3, 4, 4), (4, 5, - 4), (3, 5, 1), (5, 6, 2), (6, 7, - 2), (5, 7, 1)}

- a. Show a simulation of Dijkstra's algorithm to find the *shortest path costs* from vertex 1 to all others. You can use the following pseudocode as reference.

Note: Dijkstra's algorithm inserts all vertices in a priority-queue/min-heap at the beginning and then repeatedly extracts the vertex with minimum distance. It assumes, whenever a vertex is extracted from the queue, its shortest path cost from the source is finalized.

```
DIJKSTRA( $G, s$ )
1  for each  $v \in V[G]$ 
2      do  $d[v] \leftarrow \infty$ 
3       $\pi[v] \leftarrow \text{NIL}$ 
4   $d[s] \leftarrow 0$ 
5   $S \leftarrow \emptyset$ 
6   $Q \leftarrow V[G]$ 
7  while  $Q \neq \emptyset$ 
8      do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
9       $S \leftarrow S \cup \{u\}$ 
10     for each vertex  $v \in \text{Adj}[u]$ 
11         do if  $d[v] > d[u] + w(u, v)$ 
12             then  $d[v] \leftarrow d[u] + w(u, v)$ 
13                  $\pi[v] \leftarrow u$ 
```

- b. Explain why the algorithm could not find the correct shortest path costs for some of the vertices.
- c. Imagine you have a modified version of Dijkstra's algorithm with the following changes:
- At the beginning, it inserts only the source vertex in the priority-queue.
 - Whenever the distance of a vertex is updated, it inserts that vertex into the priority-queue.
 - It does not assume that extracting a vertex from the queue means that its shortest path cost is finalized.

Will this modified Dijkstra's algorithm be able to find the correct shortest path costs in the graph given above? Explain with reasoning or a simulation.

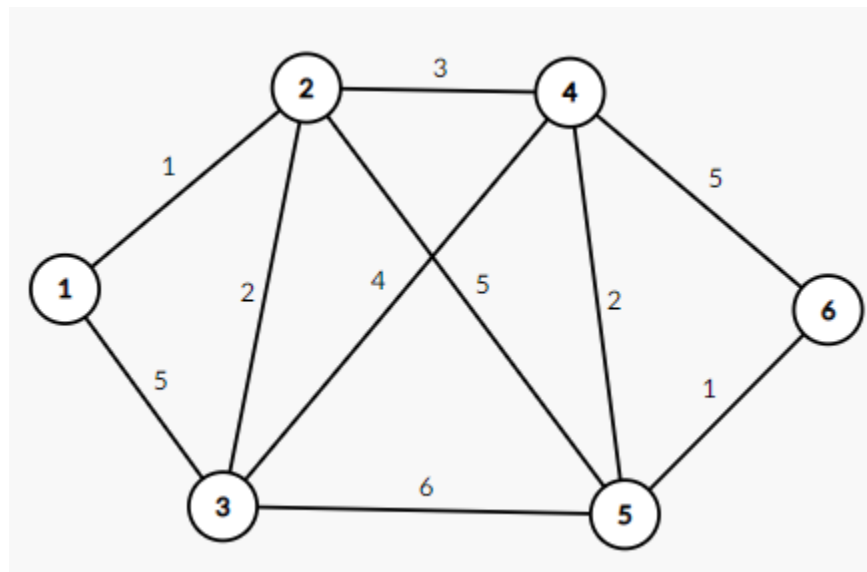
- d. Analyze the time-complexity of this modified algorithm.

Problem #4: A divide-and-conquer MST algorithm?!

Your friend Mriaslun loves the divide and conquer strategy! He recently invented a divide-and-conquer algorithm to find the Minimum Spanning Tree of a given undirected weighted graph. The algorithm is quite simple, just having 4 steps in it:

1. Divide the set of vertices into two equal halves (one side can have an extra, like we do in merge sort). Assume that this divides the graph in three parts:
 - a. G_{left} which contains all the vertices in the left half and the edges between them
 - b. G_{right} containing the vertices in the right half, and the corresponding edges
 - c. $G_{remains}$ containing the edges where one vertex is from the left half, and another one is from the right half
2. Use a recursive approach (repeating steps 1 to 4) to find the MST of G_{left} , let's call it MST_{left}
3. Do the same to find MST_{right}
4. Now you only need to select one edge from $G_{remains}$ to connect MST_{left} to MST_{right} . So you pick the one with the lowest weight value and you have the MST of the original graph!

Let's see an example.



In G_{left} , we have three vertices $\{1, 2, 3\}$ and three edges $\{(1, 2), (1, 3), (2, 3)\}$

In G_{right} , we have vertices $\{4, 5, 6\}$ and edges $\{(4, 5), (4, 6), (5, 6)\}$

And $G_{remains}$ is defined by the edges $\{(2, 4), (2, 5), (3, 4), (3, 5)\}$

If we can find MST_{left} and MST_{right} separately, then we can connect them with the minimum weight edge from $G_{remains}$, which is (2, 4). And we can use the same divide and conquer approach recursively to know that MST_{left} consists of the edges {(1, 2), (2, 3)} and MST_{right} contains the edges {(4, 5), (5, 6)}.

So, the MST of the original graph is {(1, 2), (2, 3), (2, 4), (4, 5), (5, 6)} which is the correct answer!

Now your job is to prove that this algorithm will not always give a correct answer. Provide necessary reasonings to do that. You can also create an input graph and show that the algorithm fails or provides a wrong answer for this input.

Problem #5: Coin Change

There are a few types of banknotes available in your country's currency system. Any amount of taka that you want to give someone has to be constructed using these notes. Suppose you are given an infinite supply of these banknotes. You have to find the minimum number of notes needed to construct the amount.

Now, propose an algorithm applying your knowledge on graph theory which will take the currency notes and the amount to make, K as input. Then as output, return the minimum number of notes and the value of the notes.

Examples:

Input	Output
Notes: 1, 2, 5, 10 Amount to make: 21	3 notes needed: 10, 10, 1
Notes: 3, 5, 8 Amount to make: 20	4 notes needed: 5, 5, 5, 5
Notes: 3, 5, 8 Amount to make: 7	Impossible

The following questions might help:

1. Which graph theory algorithm might be relevant here?
2. How do you convert this into a graph? What are the vertices? What are the edges?

END!
